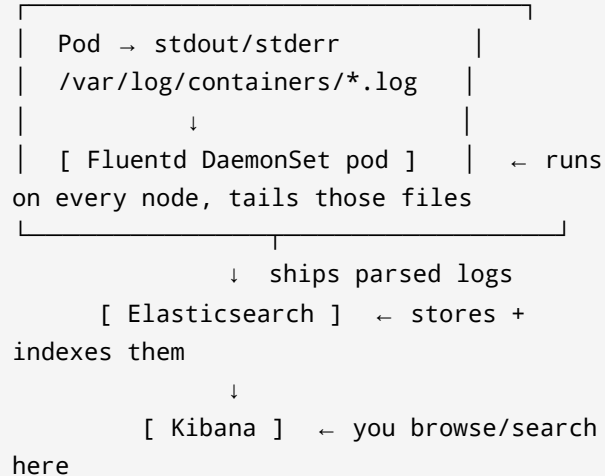


EFK Stack — Install & Configure on Kubernetes

Elasticsearch (stores/searches logs) + Fluentd (collects/ships logs) + Kibana (UI to view them).
Basic, practical setup — not a production-hardening guide, but enough to get a working pipeline you can iterate on.

0. How the pieces fit together

Every node



Fluentd = DaemonSet (one pod per node, mandatory – it needs to read files that live on each node's disk).
Elasticsearch and Kibana = normal
Deployments/StatefulSets, one set for the whole cluster.

Reminder from your earlier question: this only picks up stdout/stderr. Anything your WARs write to a file directly still needs the sidecar-tailing fix – EFK doesn't change that.

1. Namespace

```
kubectl create namespace logging
```

Keep everything here – makes cleanup and resource quotas easier later.

2. Install Elasticsearch

Using the official Elastic Helm chart (simplest reliable path):

```
helm repo add elastic  
https://helm.elastic.co  
helm repo update
```

```
helm install elasticsearch
elastic/elasticsearch \
  --namespace logging \
  --set replicas=1 \
  --set minimumMasterNodes=1 \
  --set resources.requests.cpu=500m \
  --set resources.requests.memory=1Gi \
  --set resources.limits.memory=2Gi \
  --set
volumeClaimTemplate.storageClassName=local-
path \
  --set
volumeClaimTemplate.resources.requests.stor
age=10Gi
```

- `replicas=1` / `minimumMasterNodes=1` — fine for dev/PoC. In production you'd run 3 for HA (same reasoning as etcd — quorum, no single point of failure).
- Uses your `local-path` StorageClass from earlier — works the same way here: dynamic PVC per Elasticsearch pod.
- Memory matters more than CPU for Elasticsearch — don't undersize it, it'll OOM under any real log volume.

Verify it's up:

```
kubectl get pods -n logging
kubectl get pvc -n logging
```

3. Install Fluentd (DaemonSet)

Fluentd needs RBAC permissions to read pod metadata (namespace, pod name, labels) so it can attach that context to every log line — this is what lets you filter by namespace/pod in Kibana later.

RBAC:

```
# fluentd-rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: fluentd
  namespace: logging
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: fluentd
rules:
- apiGroups: [""]
  resources: ["pods", "namespaces"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: fluentd
roleRef:
  kind: ClusterRole
```

```

name: fluentd
apiGroup: rbac.authorization.k8s.io
subjects:
- kind: ServiceAccount
  name: fluentd
  namespace: logging

```

DaemonSet (using the pre-built Fluentd image that already ships with the Elasticsearch output plugin):

```

# fluentd-daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
  namespace: logging
spec:
  selector:
    matchLabels: {app: fluentd}
  template:
    metadata:
      labels: {app: fluentd}
    spec:
      serviceAccountName: fluentd
      containers:
        - name: fluentd
          image: fluent/fluentd-kubernetes-
daemonset:v1-debian-elasticsearch7
          env:
            - {name:
FLUENT_ELASTICSEARCH_HOST, value:
"elasticsearch-
master.logging.svc.cluster.local"}

```

```

      - {name:
FLUENT_ELASTICSEARCH_PORT, value: "9200"}
      - {name:
FLUENT_ELASTICSEARCH_SCHEME, value: "http"}
      resources:
        requests: {cpu: 100m, memory:
200Mi}
        limits: {memory: 400Mi}
      volumeMounts:
        - {name: varlog, mountPath:
/var/log}
        - {name:
varlibdockercontainers, mountPath:
/var/lib/docker/containers, readOnly: true}
      volumes:
        - name: varlog
          hostPath: {path: /var/log}
        - name: varlibdockercontainers
          hostPath: {path:
/var/lib/docker/containers}

```

Apply both:

```

kubectl apply -f fluentd-rbac.yaml
kubectl apply -f fluentd-daemonset.yaml
kubectl get pods -n logging -l app=fluentd
-o wide # confirm one pod per node

```

The two `hostPath` mounts are the important part — this is literally how Fluentd reads the node-level log files we discussed earlier. If you're on containerd instead of dockershim (likely on newer clusters), the

container log path is still symlinked correctly under `/var/log/containers`, so this mount works regardless of runtime.

4. Install Kibana

```
helm install kibana elastic/kibana \
  --namespace logging \
  --set resources.requests.cpu=200m \
  --set resources.requests.memory=500Mi
```

Access it (quick way for a PoC — port-forward, don't expose publicly without auth):

```
kubectl port-forward -n logging svc/kibana-
kibana 5601:5601
```

Open `http://localhost:5601`.

For anything beyond a PoC, put an Ingress in front of it instead of port-forwarding — you already know how to do that from your ingress-controller PoC, and this is exactly where cert-manager + TLS + basic auth annotation would come in.

5. Configure Kibana to actually show logs

1. **Confirm data is arriving** — Fluentd creates a new index daily, named like `logstash-YYYY.MM.DD` by default.

```
kubectl exec -n logging -it  
elasticsearch-master-0 -- curl -s  
localhost:9200/_cat/indices?v
```

You should see index names with a growing `docs.count` . If it's empty, check Fluentd pod logs first — 90% of setup issues show up there (`kubectl logs -n logging -l app=fluentd`).

2. **Create the index pattern** — in Kibana: *Stack Management* → *Index Patterns* → *Create index pattern* → enter `logstash-*` → pick `@timestamp` as the time field.
3. **Go to Discover** — you should now see live log lines with fields like `kubernetes.pod_name` , `kubernetes.namespace_name` , `kubernetes.container_name` , `log` (the raw message). These come from the RBAC-enabled metadata enrichment Fluentd does automatically.
4. **Filter to what you care about** — e.g.
`kubernetes.namespace_name: "billing"` to

isolate just your app's logs.

6. Configuration that actually matters beyond "it's working"

- **Multiline stack traces:** by default, Fluentd treats every line as a separate log entry — a Java exception stack trace will show up as dozens of disconnected lines instead of one. Fix with a multiline parser in Fluentd's config (`concat` filter plugin, or the built-in multiline parser matching on a regex like a line starting with a timestamp). Worth doing early — this is the single most common EFK complaint from Java teams.
- **Index Lifecycle Management (ILM):** without it, Elasticsearch indices grow forever and eventually fill your disk. Set up an ILM policy (via Kibana *Stack Management* → *Index Lifecycle Policies*) to roll over and delete indices older than, say, 14–30 days.
- **Resource sizing:** Elasticsearch is the component that'll actually fall over first under real log volume — watch its memory via the Prometheus/Grafana setup you're already building, not just CPU.
- **Security:** this guide is wide open (no auth, HTTP not HTTPS) — fine for a PoC, not for anything with real data. The Elastic Helm charts support

enabling their built-in security
(`xpack.security.enabled`) with minimal extra
config when you're ready to lock it down.

7. Quick troubleshooting checklist

Symptom	Likely cause
No indices in Elasticsearch at all	Fluentd can't reach Elasticsearch — check <code>FLUENT_ELASTICSEARCH_HOST</code> matches the actual Service DNS name
Fluentd pod crash-looping	Usually RBAC — check ClusterRoleBinding was applied before the DaemonSet started
Logs from some pods missing	Those apps are writing to a file, not stdout — same issue from your Jetty/WAR question, needs the sidecar fix, not an EFK fix
Stack traces split across many entries	Multiline parsing not configured — see above

Symptom	Likely cause
Kibana shows "no matching indices"	Index pattern is <code>logstash-*</code> but Fluentd is shipping under a different index prefix — check <code>_cat/indices</code> output and match the pattern exactly